



VIKINGS™

CS 486/586

SQL Injection



PORTLAND STATE™

Origins

- **SQL Injection** was probably first documented by **Jeff Forristal** in the hacker ezine **Phrack** in **1998**.
- In the **December 1998** issue of Phrack, Forristal wrote about a series of issues with a version of Microsoft SQL server.
- When Forristal's fellow researcher told Microsoft of the problems, “their answer was, well, hilarious,” he wrote.

“According to them, what you're about to read is not a problem, so don't worry about doing anything to stop it.”



Still a Top Vulnerability

SQL Injection is still listed as a **top vulnerability** all these years later.

In 2021, SQLI was rated the number one attack on the OWASP top ten.

For the 2025 top 10, it is listed as the number 5 vulnerability.



OWASP



The **O**pen **W**orldwide **A**pplication **S**ecurity **P**roject (OWASP) is a nonprofit foundation that works to improve the security of software.

Our programming includes:

- Community-led open source projects including code, documentation, and standards
- Over 250+ local chapters worldwide
- Tens of thousands of members
- Industry-leading educational and training conferences

An open community dedicated to enabling organizations to conceive, develop, acquire, operate, and maintain applications that can be trusted. Projects, tools, documents, forums, and chapters are free and open to anyone interested in improving application security.

<https://owasp.org/about/>

OWASP Top 10

The OWASP Top 10 provides:

- A list of the **10 Most Critical** Web Application Security Risks

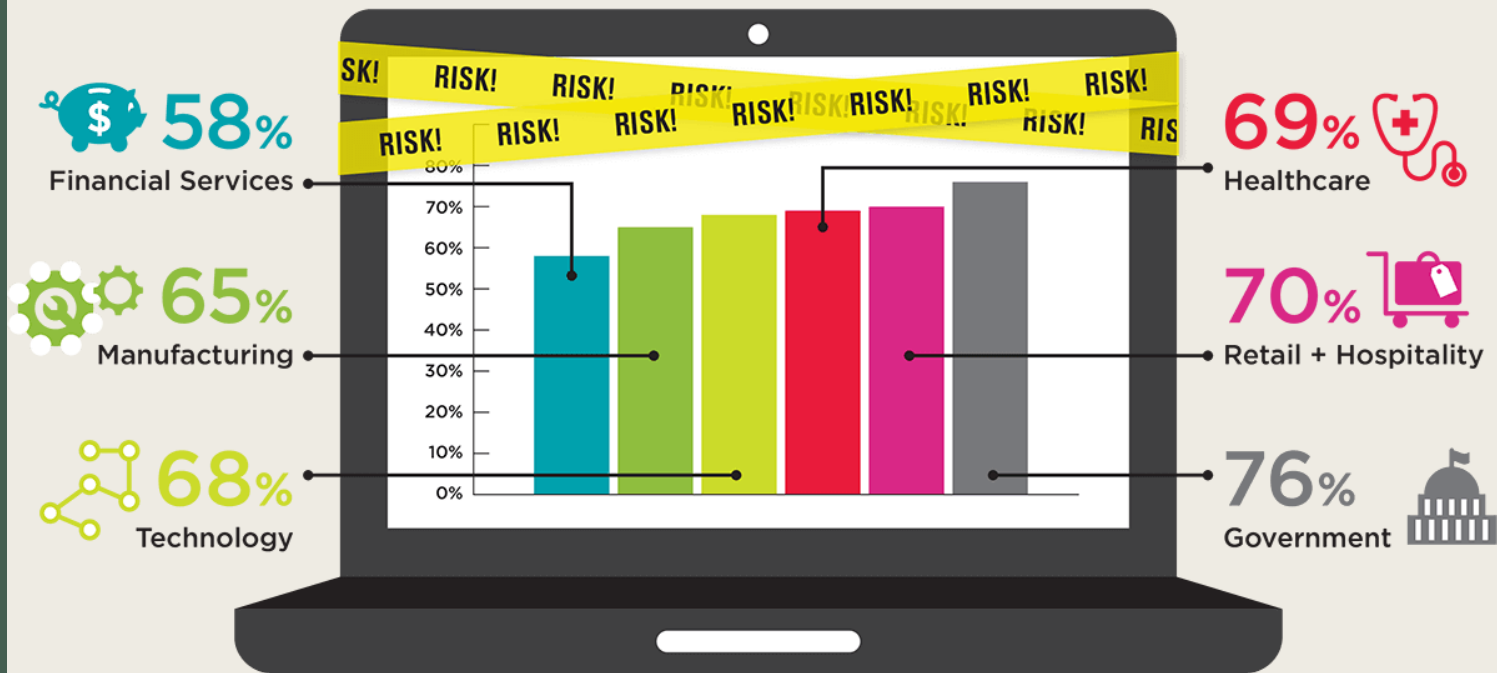
And for each Risk it provides:

- A description
- Example vulnerabilities
- Example attacks
- Guidance on how to avoid
- References to OWASP and other related resources.

Industry Applications that fail OWASP Top 10

FAILED OWASP TOP 10

How many apps fail the OWASP Top 10 upon initial risk assessment?



The data represents 208,670 application assessments submitted for analysis during the 18-month period from October 1, 2013 through March 31, 2015 by large and small companies, commercial software suppliers, open source projects and software outsourcers.

VERACODE

<https://www.veracode.com/blog/2015/10/benchmarking-your-industry-todays-software-security-landscape-sw>

OWASP Top 10 Application Security Risks - 2025

Top 10:2025 List

1. A01:2025 - Broken Access Control
2. A02:2025 - Security Misconfiguration
3. A03:2025 - Software Supply Chain Failures
4. A04:2025 - Cryptographic Failures
5. **A05:2025 - Injection**
6. A06:2025 - Insecure Design
7. A07:2025 - Authentication Failures
8. A08:2025 - Software or Data Integrity Failures
9. A09:2025 - Security Logging and Alerting Failures
10. A10:2025 - Mishandling of Exceptional Conditions

<https://owasp.org/Top10/2025/>

SQL Injection Description

An injection vulnerability is **an application flaw that allows untrusted user input to be sent to an interpreter** (e.g. a browser, database, the command line) and causes the interpreter to execute parts of that input as commands.

An application is vulnerable to attack when:

- User-supplied data is not validated, filtered, or sanitized by the application.
- Dynamic queries or non-parameterized calls without context-aware escaping are used directly in the interpreter.
- Unsanitized data is used within object-relational mapping (ORM) search parameters to extract additional, sensitive records.
- Potentially hostile data is directly used or concatenated. The SQL or command contains the structure and malicious data in dynamic queries, commands, or stored procedures.

https://owasp.org/Top10/2025/A05_2025-Injection/

SQL Injection (SQLI)

SQL Injection is a technique where **malicious** users manipulate input for web input forms or URLs, altering the SQL that the underlying application submits to the database driven web site.

The “injected” SQL commands can alter SQL statement and **compromise the security** of a web application.



Examples of SQL Injection?



Ever head of [WikiLeaks](#)?

The Panama Papers, the largest *known* data leak in history at 11.5 million sensitive documents, totaling 2.6 terabytes of data. Dating back to the 1970s and chronicling a network of [secret offshore bank accounts and shell companies](#) that politicians, world leaders, celebrities and sport executives have used to hide [millions of dollars from taxation](#) – a total of over 200k offshore entities were linked to public officials.

Examples of SQL Injection? - 2



7-Eleven breach (2007): A group of attackers used SQL injection to compromise the payment systems of several companies, including the 7-Eleven retail chain. **This breach led to the theft of over 130 million credit card numbers.** The attack was one of the largest data breaches of its time, demonstrating the immense financial and legal ramifications of SQL injection vulnerabilities.

Examples of SQL Injection? - 3



ResumeLooters campaign (2023): Between November and December 2023, the hacking group ResumeLooters compromised over 65 websites, primarily in the recruitment and retail sectors, using SQL injection and cross-site scripting (XSS) attacks. The attackers harvested over 2 million user records, including names, emails, and phone numbers. **The stolen data was later sold on various cybercrime platforms.**

Examples of SQL Injection? - 4



GhostShell university attack (2012): Team GhostShell, a hacker collective, conducted a major SQL injection attack targeting 53 universities worldwide. **They stole and published 36,000 personal records, including data from students, faculty, and staff. The attack highlighted the vulnerabilities in academic institutions' cybersecurity measures.**

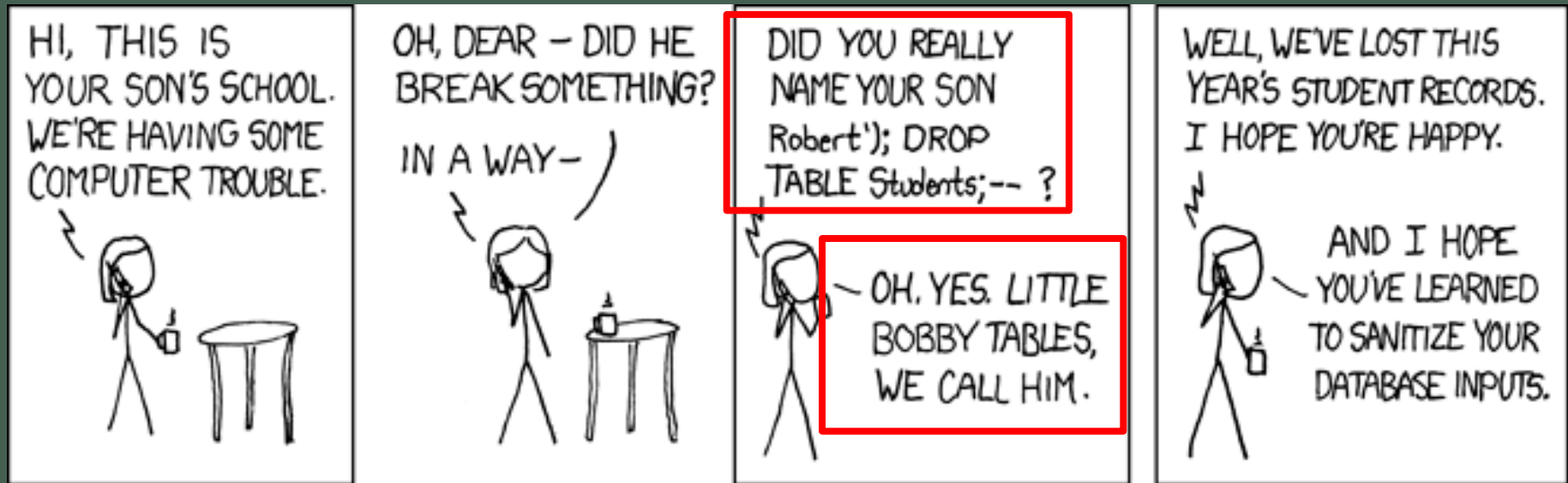
What's the Worst an Attacker can do?

- An attacker can use **SQL Injection** to **bypass authentication** or **even impersonate specific users**.
- One of SQL's primary functions is to select data based on a query and output the result of that query.
 - An SQL Injection vulnerability could allow the **complete disclosure of data** residing on a database server.
- Since web applications use SQL to alter data within a database, an attacker could use SQL Injection to **alter data stored in a database**.
 - Altering data affects data integrity and could cause repudiation issues, for instance, issues such as voiding transactions, altering balances and other records.

What's the Worst an Attacker can do? 2

- SQL is used to **delete records from a database**.
 - An attacker could use an SQL Injection vulnerability to delete data from a database.
 - Even if an appropriate backup strategy is employed, deletion of data could affect an application's availability until the database is restored.
- Some database servers are configured to allow arbitrary execution of operating system commands on the database server.
 - **Given the right conditions, an attacker could use SQL Injection as the initial vector in an attack of an internal network that sits behind a firewall.**

Poor Little Bobby





How SQL Injection Works

In order to run malicious SQL queries against a database server, an attacker must first **find an input path** in the web application that is part of a SQL query.

For an SQL Injection attack to take place, the vulnerable website needs to **directly include user input in a SQL statement**.

An attacker inserts a text that will be included as part of the SQL query and run on the database server.

SQL Injection Based on the Always True

UserId:

An attacker can enter a username like this.

Server Result

```
SELECT * FROM Users WHERE UserId = 105 or 1=1
```

The SQL above is valid. It will return all rows from the **users** table, since **WHERE 1=1** is **always true**.

Does the example above seem dangerous? If the **users** table contains names and passwords, then absolutely it is dangerous.

This is an example of a common web form construction, used to validate user login to a web site

User Name:

Password:

A user enters login information into web form input boxes.

Server Code

```
uName = getRequestString("UserName");  
uPass = getRequestString("UserPass");  
  
sql = 'SELECT * FROM Users WHERE Name =' + uName + ' AND Pass =' + uPass + ''
```

The web application **concatenates** the raw entered user name and password with some additional SQL.

Result

```
SELECT * FROM Users WHERE Name ="John Doe" AND Pass ="myPass"
```

The generated SQL string is executed by the database server.

A **malicious** hacker might get access to user names and passwords in a database by simply inserting " or ""=" into the user name or password text box

User Name:

" or ""="

Password:

" or ""="

The code at the server will create a valid SQL statement like this:

Result

```
SELECT * FROM Users WHERE Name ="" or ""="" AND Pass ="" or ""=""
```

The result SQL is valid. It will return every row from the table **users**, since **WHERE** ""="" is always true.



Batched SQL Statements

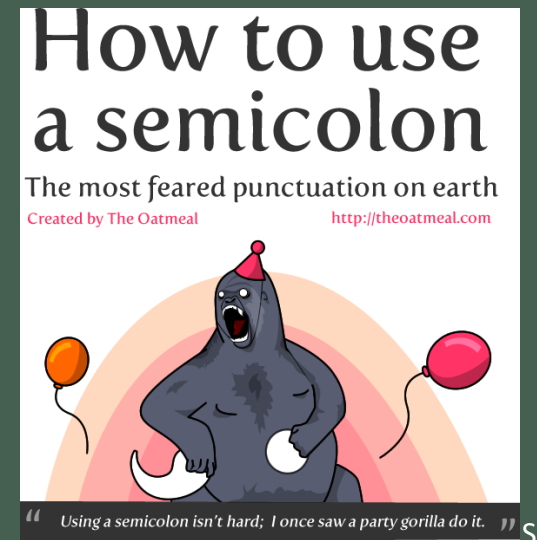
Most databases support batched SQL statements, statements separated by semicolon.

For example:

```
select * from Vendors; select * from Invoices;
```



Note the semicolon separating the SQL statements.



Batched SQL Statements - 2

Consider the following input and generated SQL

User id:

105; DROP TABLE Suppliers

The code at the server would create a valid SQL statement like this:

Result

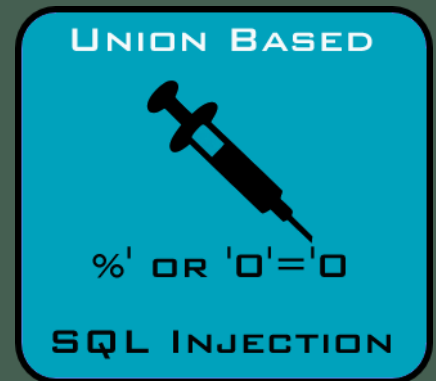
```
SELECT * FROM Users WHERE UserId = 105; DROP TABLE Suppliers
```

The generated SQL string is executed by the database server.

Since many database providers allow you to execute OS commands from within SQL, this is a way to embed OS commands into the SQL.

SQL Injection Using UNION

- UNION-based attacks allow the tester to easily extract information from the database.
- The **UNION** operator can only be used if both queries have the exact same structure, the attacker must craft a SELECT statement similar to the original query.
- In order to do this, a valid table name must be known but it is also necessary to determine the number of columns in the first query and their data type.



SQL Injection Using UNION - 2

Suppose for our examples that the query executed from the server is the following:

```
SELECT Name, Phone, Address FROM Users WHERE Id=$id
```

We will set the following `$id` value:

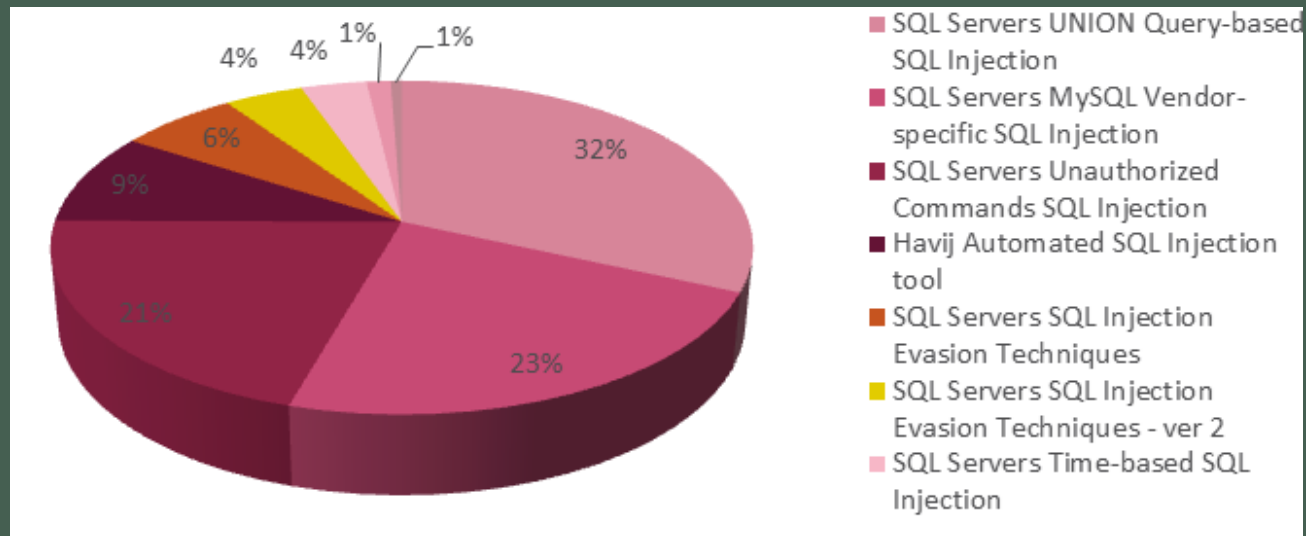
```
$id=1 UNION ALL SELECT creditCardNumber,1,1 FROM  
CreditCardTable
```

We will have the following query:

```
SELECT Name, Phone, Address FROM Users WHERE Id=1  
UNION ALL SELECT creditCardNumber,1,1 FROM  
CreditCardTable
```


Methods of SQL Injection

- Injected through user input.
- Injection through cookie fields contains attack strings.
- Injection through Server Variables.
- Second-Order Injection where hidden statements are to be executed at another time by another function.



Types of SQL Injection



There are a LOT of types of SQL Injection.

- In-band SQLi (Classic SQLi)
- Error-based SQLi
- Union-based SQLi
- Inferential SQLi (Blind SQLi)
- Boolean-based (content-based) Blind SQLi
- Time-based Blind SQLi
- Out-of-band SQLi
- ...

Different organizations have different classifications and differing terminologies. This list comes from Acunetix.

SQL Injection Prevention Cheat Sheet



OWASP
Cheat Sheets

Last revision (mm/dd/yy): **09/11/2017**

Introduction

[hide]

- 1 Introduction
- 2 Primary Defenses
 - 2.1 Defense Option 1: Prepared Statements (with Parameterized Queries)
 - 2.2 Defense Option 2: Stored Procedures
 - 2.3 Defense Option 3: White List Input Validation
 - 2.4 Defense Option 4: Escaping All User-Supplied Input

Option 2.4: STRONGLY DISCOURAGED:
Escaping All User Supplied Input

Prevention

1. The use of **prepared statements** with variable binding (aka **parameterized queries**) is how all developers should first be taught how to write database queries for use in applications.
2. Use of Stored Procedures
3. White List Input Validation
4. Escaping all User Supplied Input
5. Least Privilege

1) Prepared Statements (with Parameterized Queries)

- **Prepared statements** ensure that an attacker is not able to change the intent of a query, even if SQL commands are inserted by an attacker.



```
string sql =  
    "SELECT * FROM Customers WHERE CustomerId = @CustomerId";  
SqlCommand command = new SqlCommand(sql);  
command.Parameters.Add(new SqlParameter("@CustomerId"  
    , System.Data.SqlDbType.Int));  
command.Parameters["@CustomerId"].Value = 1;  
  
command.ExecuteReader();
```

2) Stored Procedures

- Stored procedures **are not always safe** from SQL injection.
- Stored procedure programming constructs have the similar effect as the use of parameterized queries.



```
using (SqlConnection con = new SqlConnection(dc.Con))
{
    using (SqlCommand cmd = new SqlCommand("SP_ADD", con))
    {
        cmd.CommandType = CommandType.StoredProcedure;
        cmd.Parameters.AddWithValue("@FirstName", txtfirstname);
        cmd.Parameters.AddWithValue("@LastName", txtlastname);
        con.Open();
        cmd.ExecuteNonQuery();
    }
}
```

3) White List Input Validation

- Various parts of SQL queries are not legal locations for the use of bind variables, such as the names of tables or columns, and the sort order indicator (ASC or DESC).
- In such situations, input validation or query redesign is the most appropriate defense.
- Note, **this is a symptom of poor design** and a full re-write should be considered if time allows.

If user parameter values are used to make different table names or column names, then the **parameter values should be mapped** to the legal/expected table or column names to make sure invalidated user input doesn't end up in the query.

```
string tableName;  
switch (PARAM) :  
    case "Value1": tableName = "fooTable";  
                    break;  
    case "Value2": tableName = "barTable";  
                    break;  
    ...  
    default      :  
        throw new InputValidationException(  
            "unexpected value provided for table name");
```


4) Escaping All User Supplied Input

- This technique should only be used as a **last resort**, when none of the other 3 top methods are feasible.
- Input validation is a better choice as **this methodology is frail** compared to other defenses and it cannot guaranteed it will prevent all SQL Injection in all situations.
- It is very **database specific** in its implementation.
- Its usually **only recommended as a retrofit** to legacy code when implementing input validation isn't cost effective.

Escaping All User Supplied Input - 2

- Each DBMS supports one or more character escaping schemes specific to certain kinds of queries.
- If you escape all user supplied input using the proper escaping scheme for the database you are using, the DBMS will not confuse that input with SQL code written by the developer, thus avoiding any possible SQL injection vulnerabilities.

Escaping All User Supplied Input - 3

- The OWASP Enterprise Security API (ESAPI) is a **free, open source, web application security control library** that makes it easier for programmers to write lower-risk applications.
- The ESAPI libraries are designed to make it easier for programmers to retrofit security into existing applications.
- The ESAPI libraries also serve as a solid foundation for new development.



5) Least Privilege

- To minimize the potential damage of a successful SQL injection attack, you should **minimize the privileges** assigned to every database account in your environment.

- **Do not assign DBA or admin** type access rights to your application accounts.

5) Least Privilege - 2

When I create a database, I create at least 3 accounts to go with it.

1. **A read only account.** This account can select data from tables, but not make any changes to any data in the tables. It cannot create or drop any objects in the database.
2. **A read/write account.** This account can select-delete-update data from tables. It cannot create or drop any objects in the database.
2. **A db management account.** This account can select-delete-update data from tables. It can also create or drop any object in the database.

5) Least Privilege - 3

- Where possible, you should only allow an account to access tables that it actually needs to access.
- Try to avoid just giving access to all tables.
- Do not use the db management account in your user accessible application just because that makes it easier and you've been too lazy to figure out what tables and permissions an account actually needs.

Other Comments on DB Accounts

- Use real passwords for the accounts.
- Don't just give an account a 'abc123' password because you find it hard to remember another password.
- I also give database "owners" db management permissions on the database. Application should not use the "owners" accounts.
- **Do not give the permission to create or drop databases to ANY application type account.**

OWASP References

- [OWASP SQL Injection Prevention Cheat Sheet](#)
- [OWASP Query Parameterization Cheat Sheet](#)
- [OWASP Command Injection Article](#)
- [OWASP XML eXternal Entity \(XXE\) Reference Article](#)
- [ASVS: Output Encoding/Escaping Requirements \(V6\)](#)
- [OWASP Testing Guide: Chapter on SQL Injection Testing](#)

Forewarned is to be
forearmed.





PARANOID

Just because you're paranoid, doesn't mean the world isn't out to get you.